

Agenda:

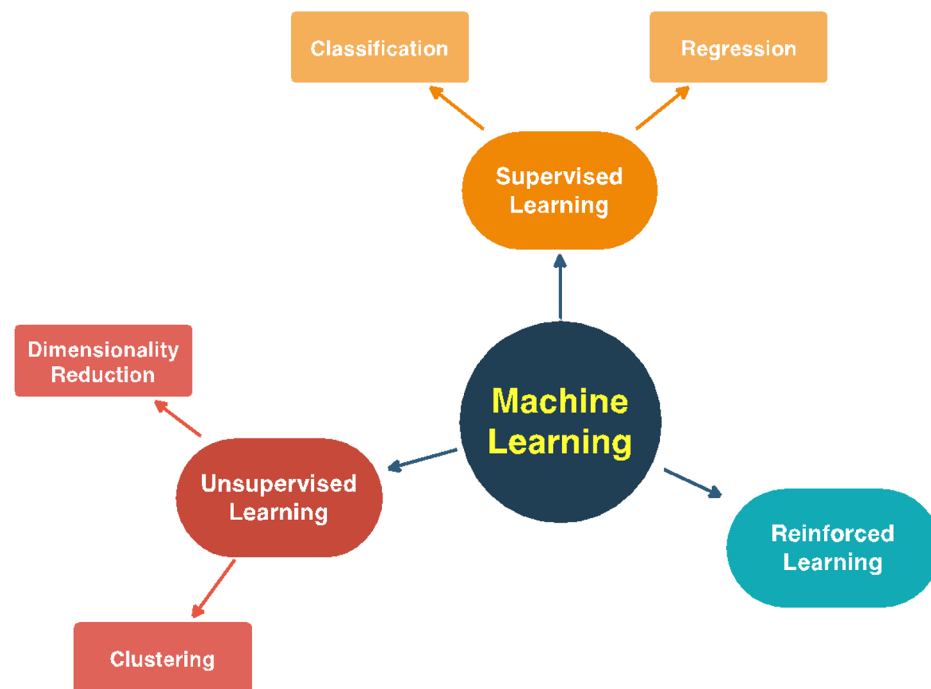
1. Activation functions
2. Cost functions

Machine Learning subcategories:

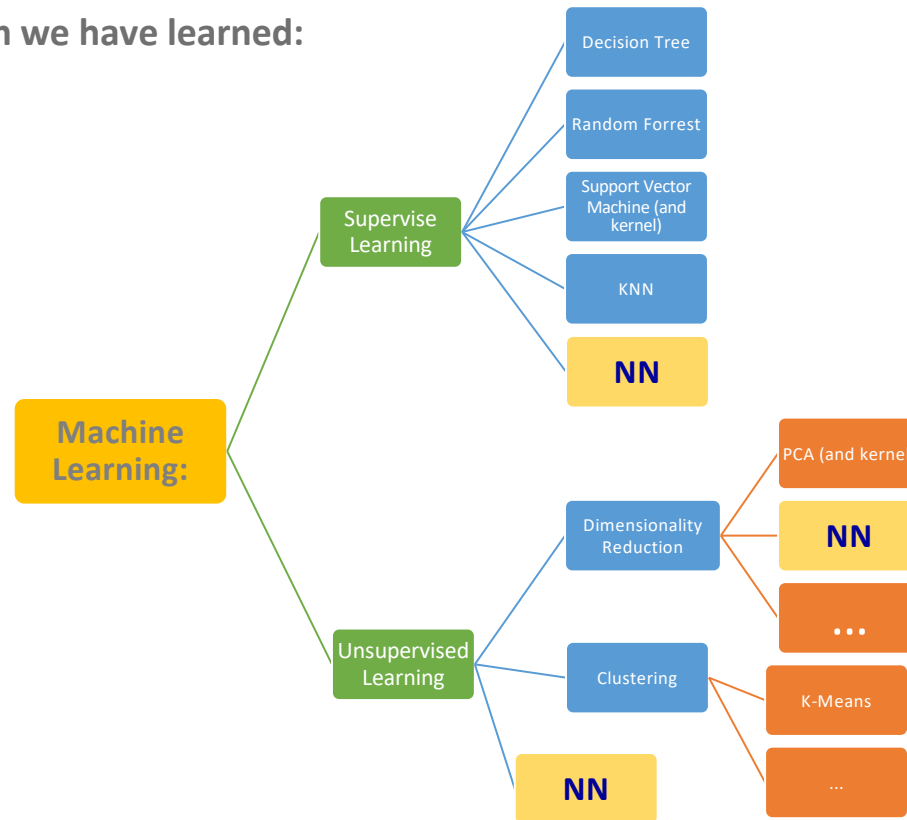
- I. Supervised Learning
- II. Un-supervised Learning
- III. Reinforced learning
- IV. Others...

Where is NN:

Almost all of them



What algorithm we have learned:

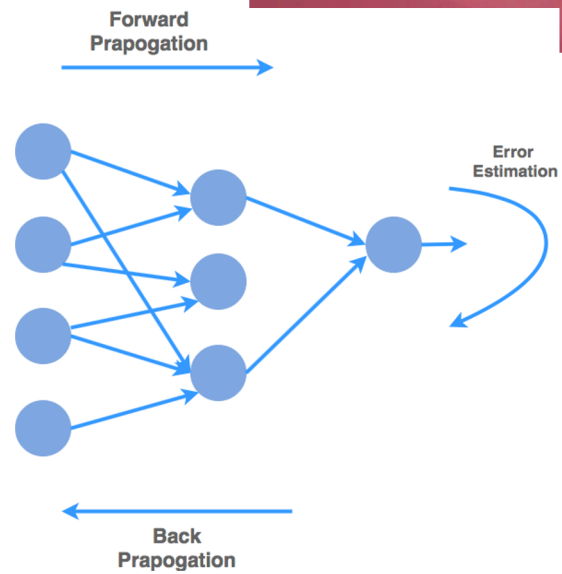


Neural Networks

Intuition behind Neural Network: How it works

Remember the tennis player?

- Forward Propagation (feedforward)
- Backward Propagation



Intuition behind Neural Network: How it works

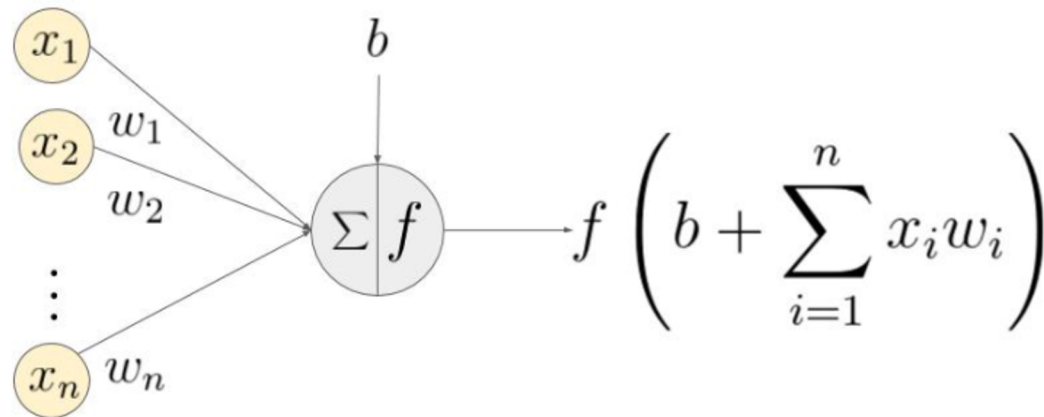
Forward Propagation:

Pre-activation: It is a weighted sum of inputs

$$h_1 = w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n + b$$

Activation: The Pre-activation is passed to a function

$$z = f(h_1)$$



Intuition behind Neural Network: How it works

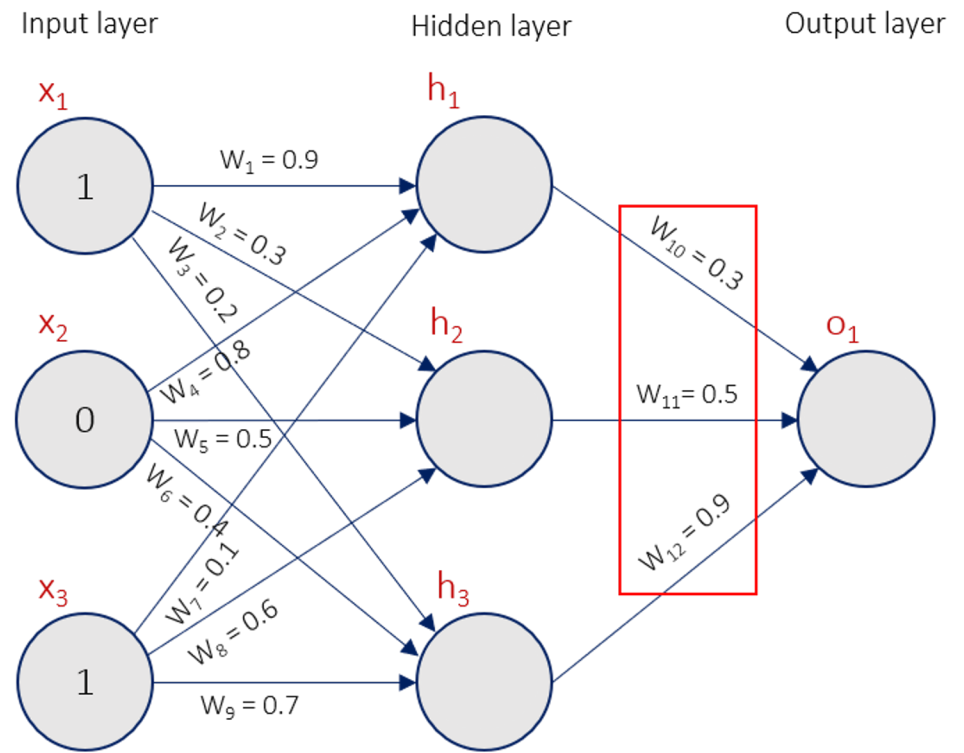
Forward Propagation:

Calculate the output node using Forward Prop.,

Activation function: Tanh in hidden layer

no activation in the output

Bias all hidden layer, 0.4, output, 0.2



Activation Functions:

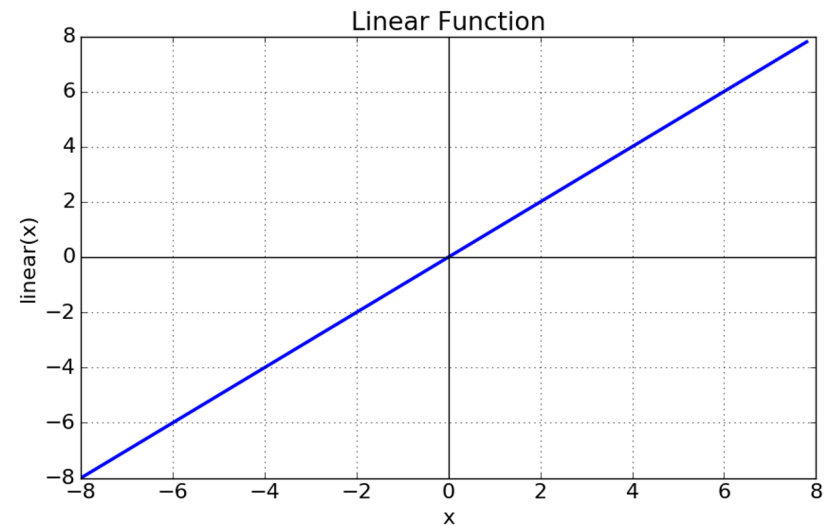
Linear (identity),

$$A(x) = cx$$

Notes:

All derivative of functions are constant

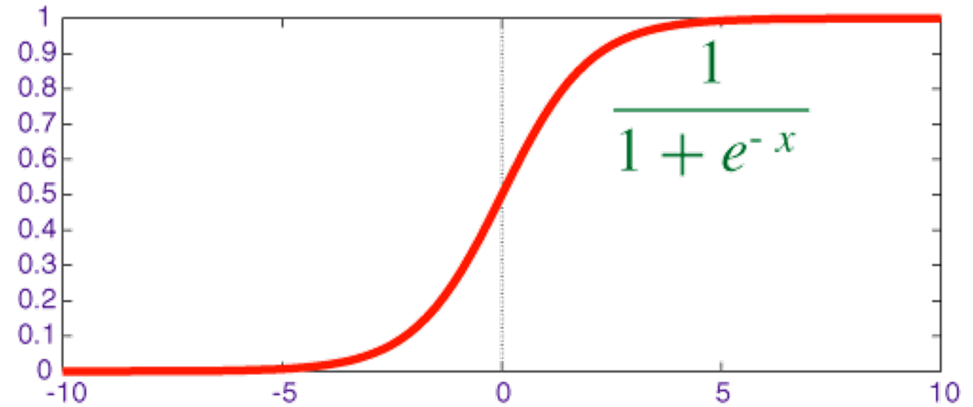
No be able to model nonlinear functions (output will be a linear function of input regardless of number of layers)



Activation Functions:

Sigmoid,

$$A(x) = \frac{1}{1+e^{-x}}$$



Mainly used for binary classification,

Pros:

Smooth gradient

Clear prediction

$0 < \text{Output} < 1$

Cons:

Computationally expensive,

Not zero centered

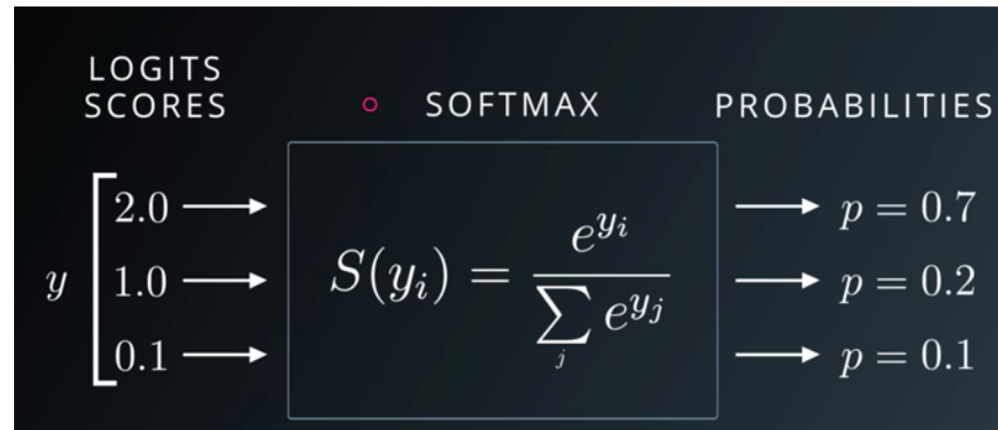
Vanishing gradient: slow learning, network will stop learning further,

Can be used in hidden and output layer.

Activation Functions:

Softmax,

$$A(x_i) = \frac{e^{x_i}}{\sum e^{x_j}}$$



Softmax is a function that convert the value of the last layer into Probability distribution, making sure the sum of probabilities are 1.

Softmax only in the Output layer.

use softmax for multi-class classification.

Activation Functions:

Softmax vs Sigmoid,

Use Softmax for classifiers when you have more than 2 classes, where as sigmoid is used for binary classifiers

Softmax return a nice probability distribution, sum of all become 1,

Sigmoid is used as an activation function in all layers,

Activation Functions:

Hyperbolic tangent, (tanh),

$$A(x) = \tanh(x)$$

Pros:

Unlike the sigmoid neuron its output is zero-centered

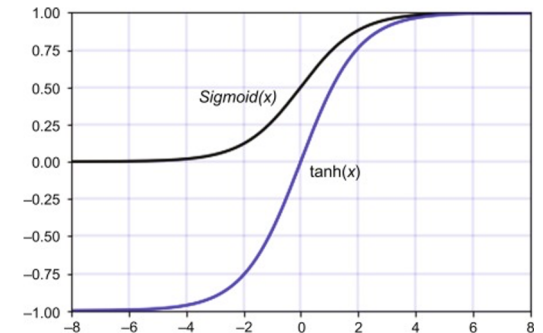
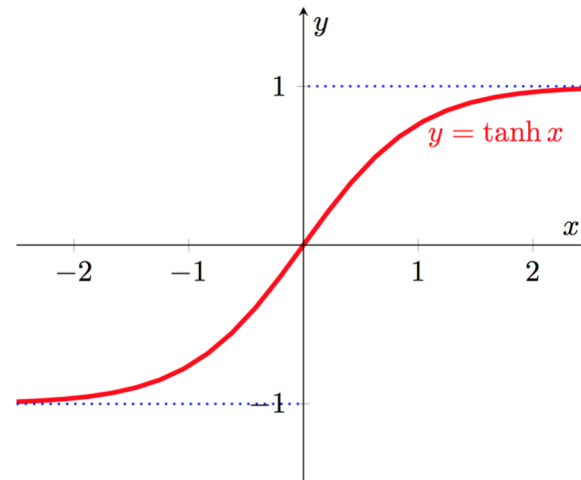
Smooth gradient (stronger than Sigmoid) ranges $(-1 - 1)$

Good for RNN

Cons:

Similar to the sigmoid neuron, its activations saturate (vanishing gradient)

Same as Sigmoid it is computationally expensive



Activation Functions:

ReLU (Rectified Linear Units),

$$A(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x > 0 \end{cases}$$

Pros:

Very cheap to compute

Generally it converge faster than sigmoid and tanh

Not much problem with vanishing gradient

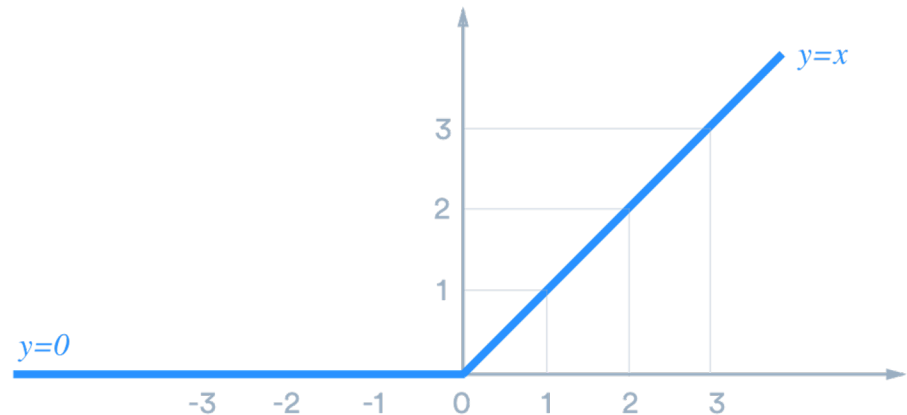
Capable of getting true zero

Cons:

Generally cant be used in RNN

All negative being zero causes the problem of dying ReLU

It may be times that it does not work well with high learning rates

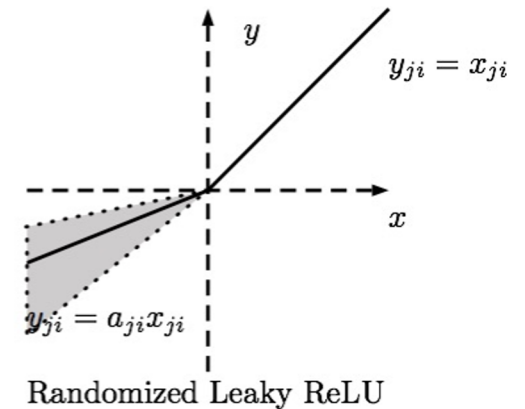
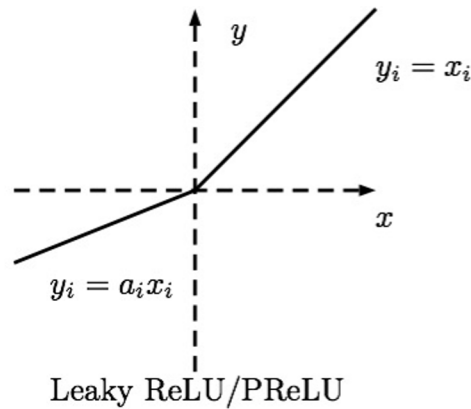


Activation Functions:

Leaky ReLU (Rectified Linear Units),

$$A(x) = \begin{cases} ax & \text{if } x < 0 \\ x & \text{if } x > 0 \end{cases}$$

With “ a ” being a small constant



Leaky ReLU is an attempt to solve the dying ReLU problem.

It solves the problem to an extent but not always successful

Activation Functions:

Other functions:

- 1. Maxout**
- 2. Modifications of ReLU**
- 3. Step**
- 4. Probit**
- 5. Logit**
- 6. ...**



Python:

Hands on exercise:

Write a python script and Plot these cost function: linear, sigmoid, tanh,

Cost Functions (or loss function):

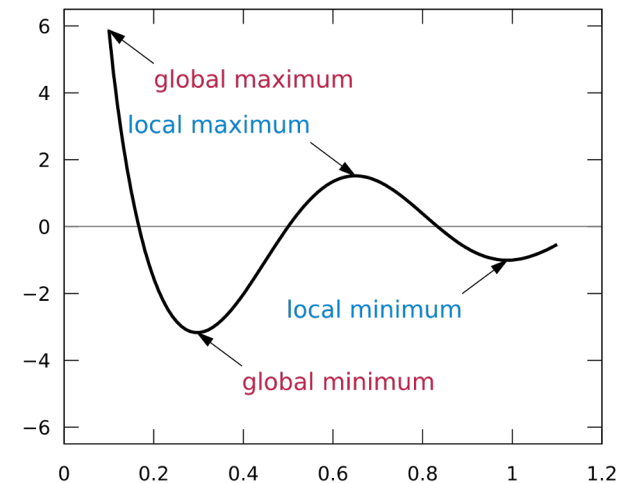
This function basically measure how well the neural network is performing, i.e. how accurate it is when doing predictions? This can be done just by comparing the predicted values with the real values (targets).

Models learn from data (experiences) by looking at the cost function, the goal is to **minimize** the cost function (find it minimum).

What are different types of cost function:

1. Mean Squared Error (MSE)
2. Cross Entropy
3. Hinge Loss
4. ...

Or any other custom cost function (for an specific purpose)



Cost Functions (or loss function):

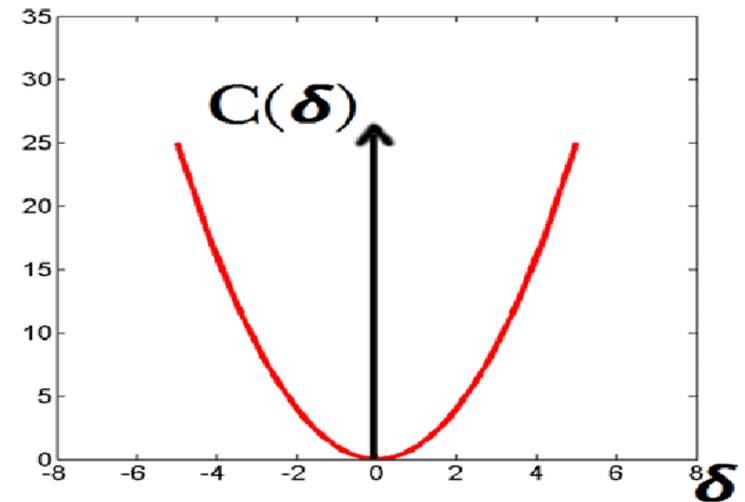
Mean Squared Error (MSE):

$$C(x) = MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

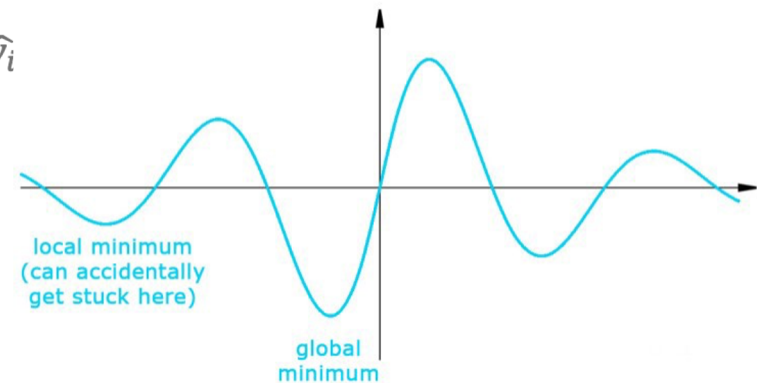
N is number of samples, y_i is the true values (target) and $\hat{y}_i$ are the predicted values (estimated).

Probably the simplest and most efficient / effective cost function.

The squared make it to take a quadratic form,



Non-Quadratic / Non-Convex



Cost Functions (or loss function):

Mean Squared Error (MSE):

Example:

Predicted	Target
2.1	3
4	3.8
5.2	6.3
1.8	1.7

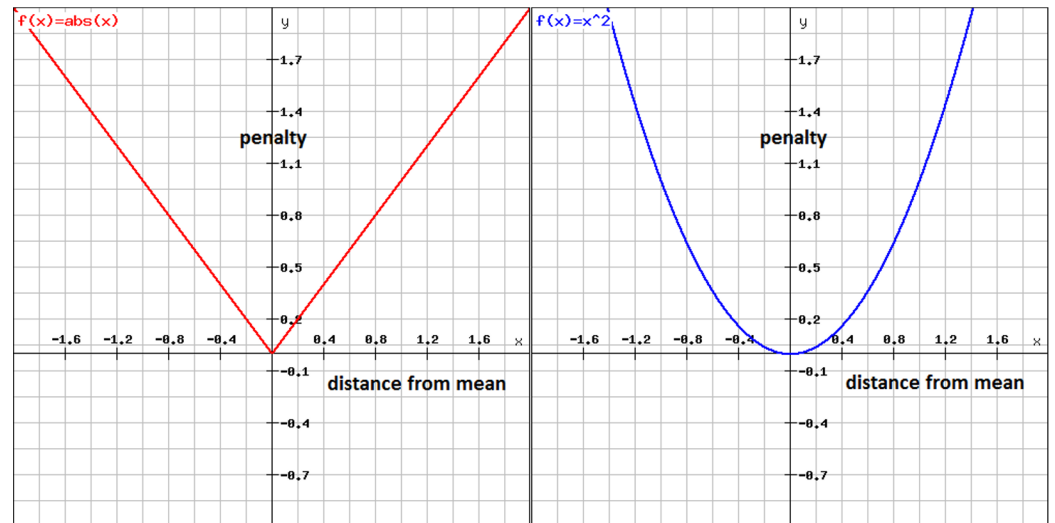
What is the cost?

Cost Functions (or loss function):

Mean absolute Error (MSE):

$$C(x) = MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

N is number of samples, y_i is the true values (target) and $\hat{y}_i$ are the predicted values (estimated).



Cost Functions (or loss function):

Mean absolute Error:

Example:

Predicted	Target
2.1	3
4	3.8
5.2	6.3
1.8	1.7

What is the cost?

Cost Functions (or loss function):

Cross Entropy:

$$C(X) = \sum_{i=1}^N p(x) \log q(x)$$

Where N is the number of samples, P is the true probability distribution and q is the estimated probability distribution.

This function is used for classification problems.

Minimizing the cost function:

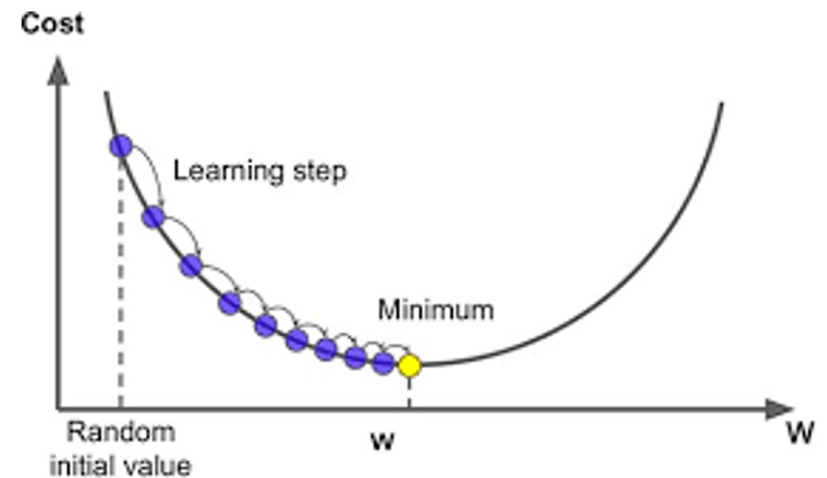
How we should minimize the cost function:

Gradient Descent:

IS an optimization algorithm capable of finding local or global minimum of a function.

It achieve this goal by iteratively moving in the direction of the sleepest descent (negative of gradient).

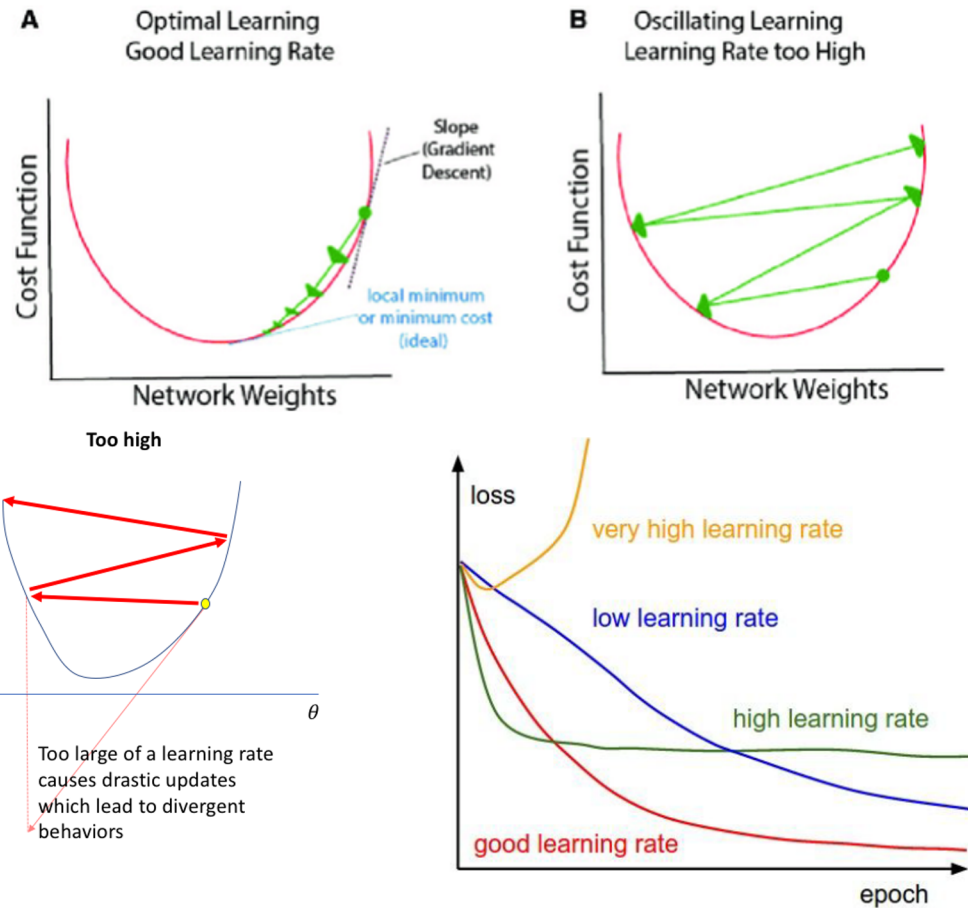
How fast or slow it moves through the minimum is controlled by the **learning rate** (think of as the tennis player).



Gradient Descent :

$$\theta_{n+1} = \theta_n + \eta \nabla(C(x_n))$$

η is the learning rate.





Python:

Install Keras TensorFlow

Do a classification NN

- Cancer classification

- source of data: <https://www.kaggle.com/shabbrikhan0015/cancer-classification>

Do a regression NN

- Cancer database, estimate tumor mean radius

Other good library is Pytorch